

Taller 01 — Foundation Models: comparación, decodificación y razonamiento

Estudiante: Paolo Arrata

Curso: MMIA 6013 — IA Generativa y Agentes, Universidad San Francisco de Quito

Fecha de las ejecuciones: 2026-09-19 y 2026-09-20

1. Introducción

Se estudió cómo cambia el comportamiento de un modelo de lenguaje cuando se mueve, de a una, cada palanca de inferencia: el muestreo (temperature, top-p, top-k), el contenido del prompt, la forma de la salida, el costo y el esfuerzo de razonamiento. Todo se hizo sobre un único problema, con un único verificador y una única función de inferencia. En total se registraron **1 594 llamadas o generaciones** (las 25 de GPT-2 local incluidas), con **20 errores**, y el gasto en APIs fue de **\$0.053**.

El resultado que más pesa es una ausencia. En este problema casi ninguna palanca movió la exactitud, porque los modelos ya acertaban todo; lo que sí se movió fue el costo. Además, varias cosas que el enunciado da por supuestas se cayeron al medirlas: un prefijo «seguro» que en GPT-2 no lo era, un nivel de esfuerzo de razonamiento que la API rechaza, y tres parámetros de muestreo que el modelo de razonamiento no acepta. Esas correcciones se cuentan en cada parte, y las limitaciones de todo esto se reúnen en las conclusiones.

2. Problema elegido

Dado un ticket de soporte, clasificarlo en exactamente una categoría: `billing`, `technical` o `account`. La salida esperada es un objeto JSON, `{"category": "billing"}`. Se eligió este problema porque la respuesta se puede verificar sin juicio humano ni un segundo modelo: se parsea el JSON y se compara la categoría con la esperada. Eso permite medir exactitud de forma determinista, repetir el mismo caso decenas de veces y comparar variantes de prompt sin que la evaluación cambie con ellas.

3. Dataset

Hay tres conjuntos, todos en `data/`:

- **10 casos oficiales** (`cases.json`, con su respuesta esperada): 4 de `billing`, 3 de `technical` y 3 de `account`. Se fijaron antes de cualquier barrido y no cambian entre modelos, temperaturas ni variantes de prompt.
- **5 casos de debug** (`cases.json`, campo `split`): solo para pruebas internas; ningún resultado del informe los usa.
- **3 casos contaminados** (`contaminated_cases.json`): uno por tipo de trampa, para la Parte 4.b.

Hay que decir de dónde salen: **todos los tickets son sintéticos**, redactados para este taller. No vienen de un sistema de soporte real, así que la exactitud que se reporta es sobre tickets fáciles y limpios, y no dice nada sobre tickets reales, que son más ruidosos y ambiguos. Cada ticket tiene una categoría correcta

inequívoca y ninguno se reutiliza como ejemplo en el prompt few-shot (hay una prueba automática que lo comprueba).

4. Metodología

Un solo camino. Cada llamada pasa por `run_case`: se construye el prompt, se llama al modelo, se verifica la respuesta con `verify_prediction`, se calcula el costo y se agrega una línea a `outputs/raw/results.jsonl`. Ese archivo es la fuente de verdad y nunca se sobrescribe; cada fila trae el modelo y el proveedor, el caso, la ejecución, los parámetros (temperature, top-p, top-k, esfuerzo), la variante de prompt, los tokens de entrada, de salida y de razonamiento, la latencia, la salida cruda, si fue correcta y el costo. Las generaciones locales de la Parte 0 (GPT-2) van en el mismo archivo con costo cero: una ejecución gratis también es una ejecución. Las tablas y gráficas de este informe se generan a partir de él (y este texto, con `scripts/build_report.py`, para no copiar números a mano). Si una llamada falla, queda registrada como error con el mensaje literal y el código HTTP.

Verificador. Distingue tres fallos: salida que no es JSON, JSON con una categoría fuera del enum, y categoría válida pero incorrecta. Se mide entonces `parse_rate` (JSON parseable), formato válido (categoría dentro del enum) y exactitud, y las tres cosas pueden diferir.

Definiciones. La exactitud de una celda es aciertos entre todas sus llamadas (un error de API cuenta como fallo). La estabilidad de un caso es la parte de sus ejecuciones que coincide con la respuesta más frecuente; la de la celda es el promedio de los casos. Una salida inválida se cuenta como una sola respuesta.

Qué mide la latencia. Es el tiempo de pared de una llamada completa sin streaming: desde justo antes de enviar la petición hasta recibir la respuesta entera, medido con `time.perf_counter()` alrededor de la llamada y con el mismo cronómetro para todos los proveedores. Incluye la red (salvo en el modelo local) y no es el tiempo hasta el primer token.

Modelos y precios. Los tres modelos de las Partes 1 y 2.a se eligieron por su id de la tabla semestral del curso. Con la clave de OpenAI, la ranura «grande» la cubre `openai_razonamiento`, y la tercera ranura, un modelo open-weight, `open_weight_pequeno`. La Parte 0 usa `base_local`.

Clave (tabla del curso)	Proveedor	Modelo	USD/1M entrada	USD/1M salida	Precio verificado
<code>propietario_economico</code>	openai	gpt-4o-mini	0.15	0.6	2026-08-26
<code>openai_razonamiento</code>	openai	gpt-5.6-luna	0.2	1.2	2026-09-18
<code>open_weight_pequeno</code>	ollama	qwen3:1.7b	0	0	— (local)
<code>base_local</code>	transformers	openai-community/gpt2	0	0	— (local)

Confirmación sobre los precios. Los precios y la fecha de verificación de cada modelo se copiaron de las filas de la tabla semestral (anexo del enunciado, generado el 2026-09-18) y no de memoria; cada fila trae su propia fecha. El costo se calcula con los tokens que reporta el proveedor, y los de razonamiento se cobran como salida.

Hay tres cosas que conviene declarar. Primero, el costo 0 de los modelos locales es la ausencia de precio por token y no cuenta el hardware; `qwen3:1.7b` se corrió con Ollama y con el razonamiento apagado (`think: false`) para compararlo como un modelo instruct normal. Segundo, `top_k` no se envía por el SDK de OpenAI de forma nativa: se manda en el cuerpo de la petición para que sea la API quien lo acepte o rechace, y no un error del cliente. Tercero, en una exploración inicial se usó `gpt-5.5`, que **no** es una fila de la tabla del curso y cuyo precio salió de la página de OpenAI; esas 27 filas se conservan en el archivo crudo con la clave `extra_gpt55`, pero ninguna tabla de este informe las usa. Las APIs no garantizan salidas

idénticas ni con temperature 0, y no se afirma lo contrario; los barridos caros son reanudables (una llamada ya registrada como correcta se omite) y la semilla es 42 donde el backend lo permite.

5. Parte 0 — GPT-2 base

Se elige el prefijo por medición, no por suposición. El plan sugiere The capital of France is como frase de alta confianza. En GPT-2 no lo es: a T=1 su token más probable es the con probabilidad **0.085** y la entropía es de **8.65 bits**, casi la de un prefijo incierto (9.33 bits). Por eso se midió la entropía a T=1 de 7 frases candidatas y se eligió la de menor entropía, Thank you very (0.10 bits; much con 0.992), el comienzo de una frase hecha. Para el prefijo incierto se usó una frase del dominio, A customer support ticket about an unexpected.

Prefijo	Entropía a T=1 (bits)	Token más probable	p	Elegido como
The capital of France is	8.65	' the'	0.085	
The quick brown fox jumps over the lazy	9.29	' '	0.056	
Once upon a	8.51	' time'	0.177	
Thank you very	0.10	' much'	0.992	alta confianza
The United States of	0.53	' America'	0.966	
Happy birthday to	7.71	' you'	0.187	
1, 2, 3, 4, 5, 6,	0.60	' 7'	0.951	
A customer support ticket about an unexpected	9.33	' problem'	0.046	baja confianza

0.a — la distribución del siguiente token. Se obtuvieron los logits del siguiente token en un solo paso hacia adelante y se calculó softmax(logits / T) con T en {0.1, 0.7, 1.0, 1.5, 2.0}. Las Figuras 1 a 3 muestran los 15 candidatos más probables y la entropía, y la tabla da la entropía en bits y el tamaño del núcleo, es decir, cuántos tokens acumulan el 90 % de la masa (top_p = 0.9 medido).

Prefijo	T	Entropía (bits)	Núcleo top-p 0.9
Thank you very	0.1	0.00	1
Thank you very	0.7	0.01	1
Thank you very	1.0	0.10	1
Thank you very	1.5	2.15	12
Thank you very	2.0	9.77	14537
A customer support ticket about an unexpected	0.1	0.95	2
A customer support ticket about an unexpected	0.7	6.39	154
A customer support ticket about an unexpected	1.0	9.33	1550
A customer support ticket about an unexpected	1.5	12.24	8364
A customer support ticket about an unexpected	2.0	13.54	15726

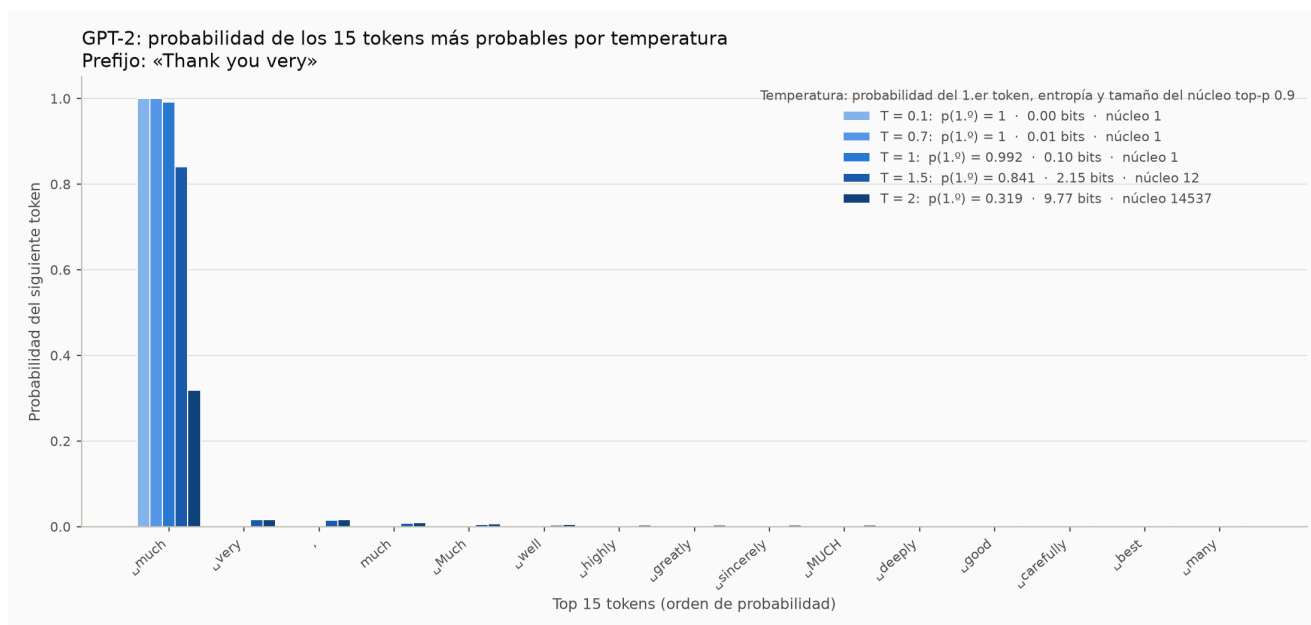


Figura 1. Probabilidad de los 15 tokens más probables para el prefijo «Thank you very», una barra por temperatura. La leyenda da la probabilidad del primer token, la entropía y el tamaño del núcleo top-p 0.9.

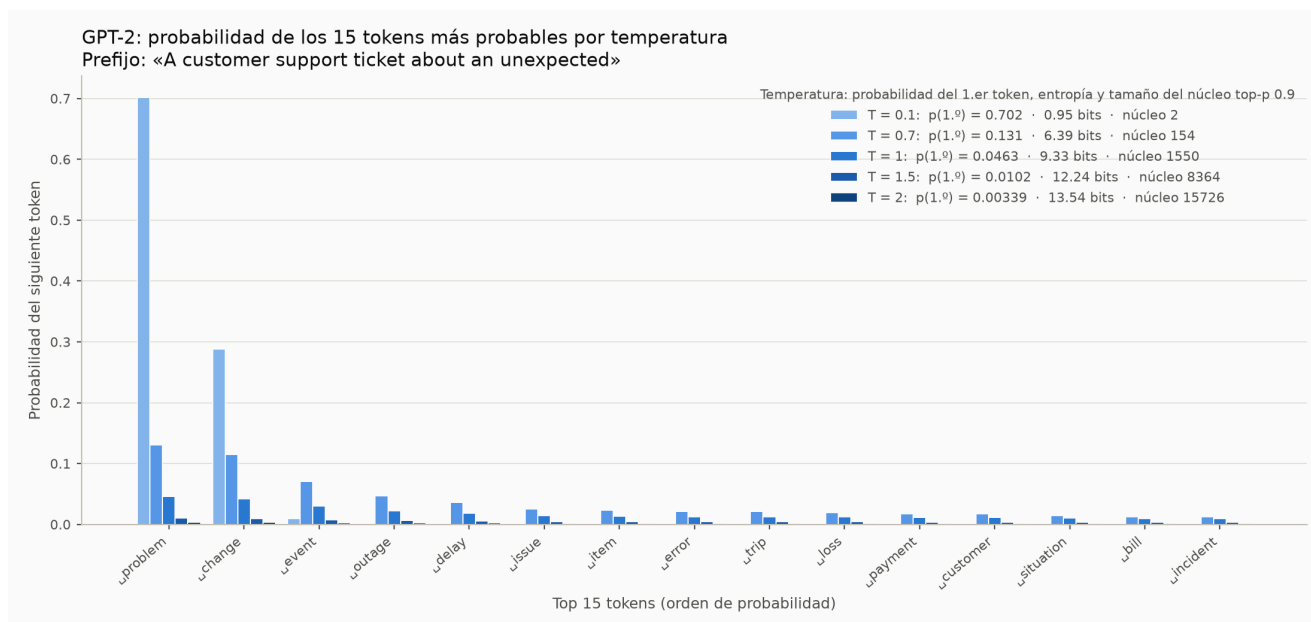


Figura 2. Lo mismo para el prefijo del dominio, «A customer support ticket about an unexpected».

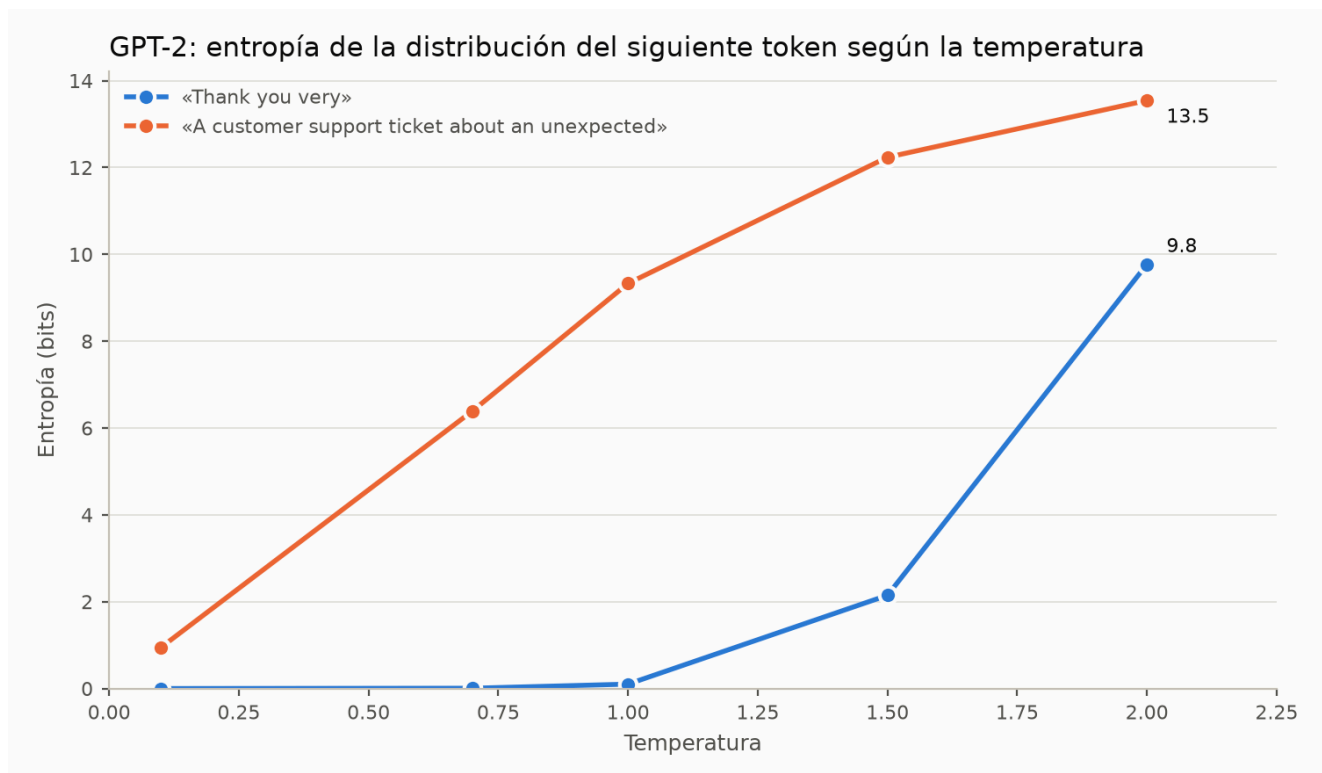


Figura 3. Entropía de la distribución del siguiente token según la temperatura, para los dos prefijos.

Qué le hace la temperatura a la distribución. La temperatura divide los logits antes del softmax, así que no cambia el orden de los tokens sino cuánto pesa cada uno. Con $T=0.1$ la masa se concentra en el primer token: la entropía de «Thank you very» cae a 0.00 bits y el núcleo es de 1 token(s); con $T=2.0$ sube a 9.77 bits y 14537 tokens. El tamaño del núcleo depende del prefijo: a $T=1$ el prefijo seguro necesita 1 token(s) para acumular el 90 % y el incierto, 1550; a $T=2.0$ los dos se acercan (14537 y 15726). Así, `top_p = 0.9` no es una cantidad fija de tokens sino una masa, y el conjunto que corresponde a esa masa cambia con el prefijo y con la temperatura.

o.b — las tres palancas (prefijo `A customer support ticket about an unexpected`, 40 tokens nuevos, semilla fijada; las salidas crudas están en el Anexo A):

- 1. El interruptor.** Con `do_sample=False` y temperature 0.2 o 1.5, la salida es **idéntica** entre las dos y respecto de greedy (A.1 y A.2). Es así porque temperature solo reescala los logits antes de muestrear, y con `do_sample=False` no se muestrea: se toma el argmax, que no cambia al dividir todos los logits por un número positivo. La librería lo avisa: «The following generation flags are not valid and may be ignored: ['temperature']. Set `TRANSFORMERS_VERBOSITY=info` for more details.». Es el estado «aceptado y no actúa» de la Parte 2.a, observado en local.
- 2. top_k=1 reproduce greedy.** Con `do_sample=True` y `top_k=1`, cinco ejecuciones con semillas 42–46 dan salidas **idénticas entre sí y iguales a greedy**. Además de comparar textos se comparó contra la distribución: los ids generados coinciden con los de greedy en **5 de 5** ejecuciones, y en **5 de 5** ejecuciones cada uno de los 40 tokens es el argmax de los logits crudos de su paso (A.3).
- 3. Dónde corta cada palanca.** Con `top-k=5` y `top-p=0.9` sobre la misma distribución ($T=1$) sobreviven conjuntos distintos, y en sentido contrario según la confianza del modelo: en el prefijo seguro top-k conserva **5** tokens y top-p **1**; en el incierto, top-k conserva **5** y top-p **1550**. Los dos prefijos sirven de ejemplo de que los conjuntos no coinciden: top-k corta por cantidad fija y top-p por masa fija (Figura 4, en escala logarítmica porque con un token dominante el resto sería invisible).

4. **Degeneración.** Con greedy a 100 tokens la generación entra en un bucle: la frase «The customer support ticket about an unexpected problem» aparece **9 veces** (A.4, sin editar). Es la degeneración por repetición que describen Holtzman et al. (2020).

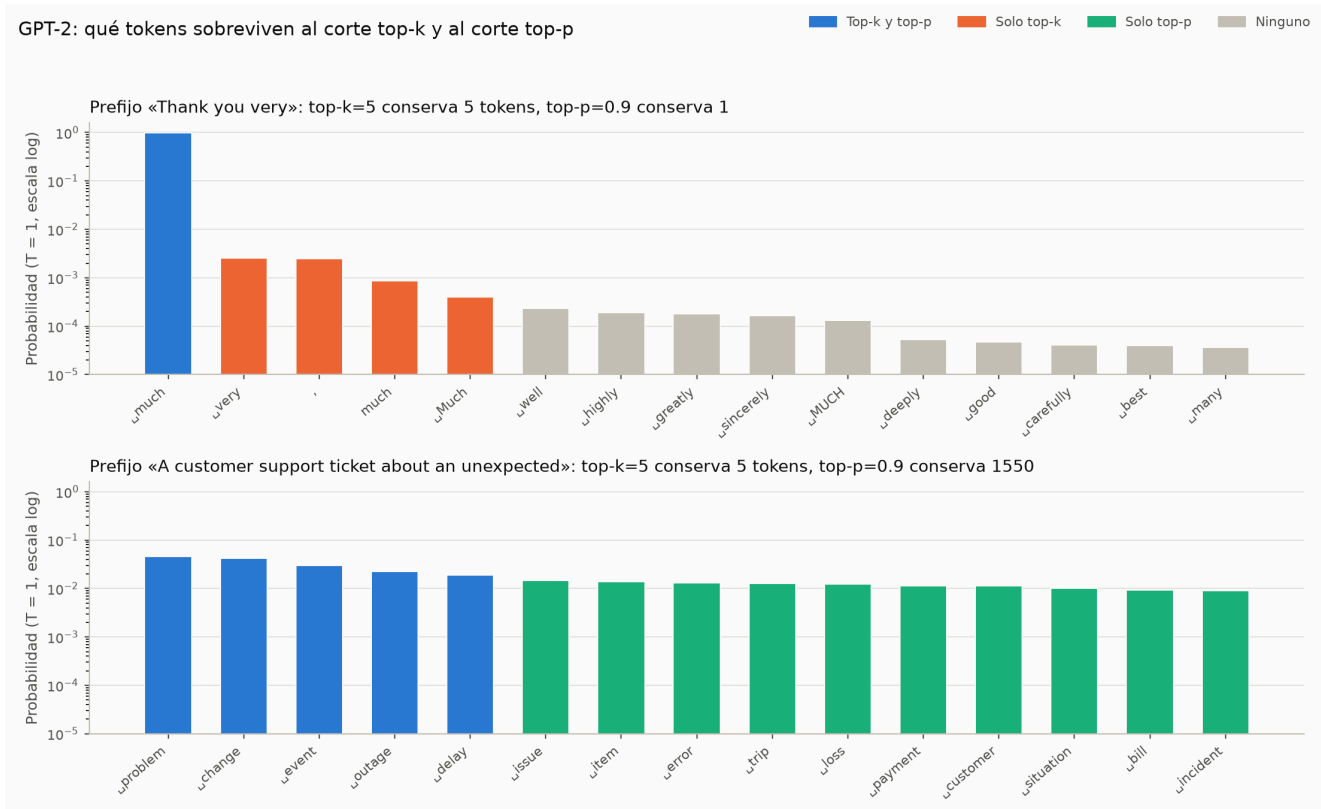


Figura 4. Qué tokens conservan el corte top-k=5 y el corte top-p=0.9 sobre la misma distribución ($T=1$), en un prefijo seguro (arriba) y en uno incierto (abajo). Escala logarítmica en el eje vertical.

Las definiciones de top-k y top-p que se usaron para contar los conjuntos se comprobaron contra los `TopKLogitsWarper` y `TopPLogitsWarper` de `transformers`, que son los que aplica `generate()`.

o.c — el límite del modelo base. Se le dio a GPT-2 la misma instrucción que se usa en las Partes 1 a 4 (el prompt base con el ticket del caso 1 al final) y su salida cruda, sin editar (60 tokens, greedy), es esta; las de los 10 casos están en el Anexo B:

```
I was charged twice for the same subscription.
I was charged twice for the same subscription.
I was charged twice for the same subscription.
I was charged twice for the same subscription.
I was charged twice for the same subscription.
I was charged
```

En lugar de obedecer, GPT-2 continúa el texto. En 4 de los 10 casos repite el ticket tal cual; en otro inventa un comando `curl` que copia el formato JSON del prompt con un valor sin sentido, y en otros divaga. Nunca produce una categoría: 0 de 10 respuestas parseables. Lo que le falta, y sí tienen los modelos de las otras partes, es lo que viene después del preentrenamiento: ajuste con instrucciones y con preferencias humanas, que enseña que un texto con forma de orden es una orden y que la respuesta debe seguir el forma-

to pedido. GPT-2 solo aprendió a predecir la palabra siguiente, y para él la instrucción es un prefijo más. Además es mucho más pequeño y antiguo, un factor que este experimento no separa del alineamiento.

6. Parte 1 — comparación de modelos

Se ejecutó el mismo prompt (el prompt base de la Parte 0.c) sobre tres modelos de la tabla del curso: `openai_razonamiento` (la ranura «grande» con la clave de OpenAI), `propietario_economico` y `open_weight_pequeno`. **Los tres acertaron los 10 casos** con JSON válido, así que el problema no los separa por exactitud; los separa el costo y la latencia.

Modelo	Exactitud	Latencia media (s)	Latencia p50 (s)	Tokens entrada	Tokens salida	Costo total (10 casos)	Precio verificado	Parse rate
gpt-4o-mini	1.00	0.70	0.70	57.1	6.0	\$0.00012	2026-08-26	1.00
gpt-5.6-luna	1.00	1.32	1.05	56.1	8.0	\$0.00021	2026-09-18	1.00
qwen3:1.7b	1.00	0.15	0.13	66.3	7.0	\$0.00000	— (local)	1.00

Las observaciones cualitativas (¿alucina?, ¿respeto el formato?, ¿es verboso?): ningún modelo produjo una categoría inexistente ni texto fuera del JSON, y los tres respetaron el formato pedido (`parse_rate` 1.00). En verbosidad no hay diferencias: 6.0, 8.0 y 7.0 tokens de salida por llamada para `gpt-4o-mini`, `gpt-5.6-luna` y `qwen3:1.7b`. `gpt-5.6-luna` es un modelo de razonamiento, pero con el esfuerzo por defecto usó en promedio 0.0 tokens de razonamiento en estos tickets. La primera llamada al modelo local tardó 3.55 s por cargar el modelo en memoria; esa fila no entra en la tabla porque se usa la última fila por caso. El detalle está en `report/part1_notes.json`.

La latencia de `qwen3:1.7b` (0.15 s) no incluye red, así que no se compara directamente con la de los modelos de pago (0.70 s y 1.32 s). `gpt-5.6-luna` costó 1.7 veces lo de `gpt-4o-mini` para el mismo resultado (\$0.00021 frente a \$0.00012 por los 10 casos).

Conclusión: qué modelo elegiría en producción. Elegiría `propietario_economico` (`gpt-4o-mini`). Con exactitud igual en los tres, decide el costo por llamada y la latencia: es el más barato de los modelos de pago (USD 0.15 / 0.6 por millón frente a 0.2 / 1.2 de `openai_razonamiento`), el más corto en salida y más rápido que el modelo de razonamiento, y no obliga a operar hardware. `open_weight_pequeno` sería mi elección si los tickets no pudieran salir de la infraestructura o si el volumen justificara el equipo: es gratis por llamada y el más rápido, pero hay que servirlo y monitorearlo. `openai_razonamiento` no aporta aquí porque casi no razona en esta tarea. Es una conclusión sobre 10 tickets fáciles: no dice cómo se compararían con tickets ambiguos ni bajo carga.

7. Parte 2.a — matriz de exposición de parámetros

Para cada uno de los tres modelos de la Parte 1 y cada parámetro se probó un valor bajo y uno alto (`temperature` 0.0 y 2.0, `top_p` 0.01 y 1.0, `top_k` 1 y 100), con 5 ejecuciones cada uno y un prompt abierto donde el muestreo sí cambia la salida. Un HTTP 200 no basta para decir que un parámetro actúa: el criterio fue que hubiera más salidas distintas con el valor alto que con el bajo, y si la API respondía 400 o 422 se registraba el rechazo con su mensaje literal. La columna «Declara la tabla» es el campo `parametros_expuestos` de la tabla semestral; la de «Observado» es lo que se midió, con la fecha de la prueba.

Modelo	Parámetro	Declara la tabla	Observado	HTTP	Salidas distintas (bajo / alto)	Fecha de la prueba
propietario_economico	temperatura	si	acepta y actúa	200	1 / 5	2026-09-19
propietario_economico	top_p	si	acepta y actúa	200	1 / 5	2026-09-19
propietario_economico	top_k	no	rechazado	400	0 / 0	2026-09-19
openai_razonamiento	temperatura	no_en_esta_fila	rechazado	400	0 / 0	2026-09-20
openai_razonamiento	top_p	no_en_esta_fila	rechazado	400	0 / 5	2026-09-20
openai_razonamiento	top_k	no	rechazado	400	0 / 0	2026-09-20
open_weight_pequeno	temperatura	si	acepta y actúa	200	1 / 5	2026-09-19
open_weight_pequeno	top_p	si	acepta y actúa	200	1 / 4	2026-09-19
open_weight_pequeno	top_k	si	acepta y actúa	200	1 / 3	2026-09-19

De las 9 celdas, **5 aceptan y actúan y 4 se rechazan**. `propietario_economico` y `open_weight_pequeno` aceptan y actúan en lo que la tabla declara, y `top_k` lo rechaza la API de OpenAI en sus dos modelos, como declara la tabla. Los mensajes literales de cada celda rechazada:

- `propietario_economico`, `top_k` (HTTP 400, 2026-09-19): «Unrecognized request argument supplied: `top_k`»
- `openai_razonamiento`, `temperature` (HTTP 400, 2026-09-20): «Unsupported value: 'temperature' does not support 0.0 with this model. Only the default (1) value is supported.»
- `openai_razonamiento`, `top_p` (HTTP 400, 2026-09-20): «Unsupported parameter: 'top_p' is not supported with this model.»
- `openai_razonamiento`, `top_k` (HTTP 400, 2026-09-20): «Unknown parameter: 'top_k'. Did you mean 'top_p'?»

Discrepancia con la tabla, con fecha (2026-09-20). Para `openai_razonamiento` la tabla declara `temperature` y `top_p` como `no_en_esta_fila`: el proveedor los expone en otra fila y los rechaza en ésta. Con la clave del curso se comprobó contra `api.openai.com` y el resultado es más matizado: la API los rechaza, pero para `temperature` el mensaje dice que el único valor admitido es el por defecto (1), y `top_p=1.0`, que es el valor por defecto, se aceptó (celda `top_p`: 0 / 5 salidas distintas, bajo / alto), mientras que 0.01 se rechazó. Es decir, se comporta como `solo_valor_por_defecto`. Esto también cierra el hueco que plantea el enunciado entre la documentación de Azure y la de OpenAI: la API de OpenAI rechaza estos parámetros para este modelo. `top_k` se rechaza como parámetro desconocido, como declara la tabla. Como comprobación adicional, el modelo `gpt-5.5` (fuera de la tabla del curso) rechazó también 3 de sus 3 celdas.

Sobre la ortografía: OpenAI rechaza los parámetros desconocidos con un 400, así que un nombre mal escrito no pasaría por «aceptado»; en Ollama, que acepta claves inventadas con 200, los tres nombres (`temperature`, `top_p`, `top_k`) se comprobaron con dos valores extremos y varias ejecuciones y sus efectos se ven en la columna de salidas distintas.

Conclusión de la Parte 2.a. Un parámetro documentado no es un parámetro que actúa, y uno que se rechaza depende del modelo y no del proveedor: dos modelos de OpenAI dieron resultados opuestos para `temperature` y `top_p`. La matriz describe el estado de un servicio en una fecha; por eso lleva fecha.

8. Parte 2.b — barrido de decoding

Se barrieron 15 celdas de temperature $\times$ top-p sobre propietario_economico (la matriz de 2.a lo marca «aceptado y actúa» en los dos parámetros; 10 casos $\times$ 5 ejecuciones por celda, 750 llamadas) y, aparte, top-k sobre open_weight_pequeno (200 llamadas), porque OpenAI no expone top_k. Para cada celda se registró la exactitud media sobre los 10 casos, la estabilidad (de las 5 ejecuciones de un mismo caso, cuántas dieron la misma respuesta, promediada) y la longitud media de la salida en tokens.

Temperature	Top-p	Llamadas	Exactitud	Estabilidad	Tokens de salida (media)
0.0	0.5	50	1.00	1.00	6.00
0.0	0.9	50	1.00	1.00	6.00
0.0	1.0	50	1.00	1.00	6.00
0.3	0.5	50	1.00	1.00	6.00
0.3	0.9	50	1.00	1.00	6.00
0.3	1.0	50	1.00	1.00	6.00
0.7	0.5	50	1.00	1.00	6.00
0.7	0.9	50	1.00	1.00	6.00
0.7	1.0	50	1.00	1.00	6.00
1.0	0.5	50	1.00	1.00	6.00
1.0	0.9	50	1.00	1.00	6.00
1.0	1.0	50	1.00	1.00	6.00
1.5	0.5	50	1.00	1.00	6.00
1.5	0.9	50	1.00	1.00	6.00
1.5	1.0	50	1.00	1.00	5.98

En las 15 celdas la exactitud mínima fue 1.00 y la estabilidad mínima 1.00, con 0 errores. La única diferencia visible fue una salida con JSON compacto en la celda de temperatura 1.5 y top-p 1.0 (5.98 tokens de salida en media), con la misma categoría. Con temperatura 0 las 5 ejecuciones de cada caso coincidieron en todas las celdas (estabilidad 1.00): no se observó la irreproducibilidad que el enunciado advierte que puede ocurrir, pero eso no autoriza a suponerla ausente en otro momento o modelo. El barrido de top-k, a temperatura fija de 0.7 y con un baseline greedy:

top-k	Temperatura	Llamadas	Exactitud	Estabilidad	Tokens de salida (media)	Coincide con greedy
— (greedy)	0.0	50	1.00	1.00	7.0	—
1	0.7	50	1.00	1.00	7.0	1.00
5	0.7	50	1.00	1.00	7.0	—
40	0.7	50	0.98	0.98	7.0	—

Con top-k=1 la salida coincide con la del baseline greedy en una proporción de 1.00 de los casos, así que top-k=1 reproduce greedy (misma salida en las 5 ejecuciones). Con top-k=40 hubo un fallo en 50 llamadas (exactitud 0.98).

Qué cambia al pasar de una distribución a una exactitud, respecto de la Parte 0.b: allí se compara una distribución completa sobre 50 257 tokens y los cortes son visibles; aquí lo que se mide es una etiqueta de un solo token con la masa muy concentrada, y por eso ningún corte cambia la respuesta: en el prefijo seguro de la Parte 0 la entropía es casi cero hasta $T=1$. No contradice la Parte 2.a, donde el texto era libre y los parámetros sí cambiaron la salida. Un fallo aislado a top-k=40 y una variación de formato en 950 llamadas son eventos únicos; con estos conteos no alcanzan para hablar de tendencia.

Conclusión. La temperatura divide los logits antes del softmax: por debajo de 1 acentúa las diferencias y concentra la masa en el token más probable, y por encima de 1 las atenúa y reparte la masa entre más tokens (en la Parte 0 la entropía del prefijo seguro pasó de 0.00 a 9.77 bits entre $T=0.1$ y $T=2.0$). Para mi tarea, extraer una etiqueta, elegiría temperatura 0 con top-p 1.0: las 15 celdas dieron exactitud y estabilidad

1.00, así que no hay ganancia en aceptar variación, y T=0 es la rama argmax, la más simple de justificar. No es una promesa de reproducibilidad en una API; aquí las 5 ejecuciones coincidieron y no se puede extrapolar. Si la tarea fuera generar tres titulares alternativos elegiría temperatura cerca de 0.7 con top-p 0.9: en la Parte 2.a un ajuste bajo dio 1 salida distinta en 5 ejecuciones y uno alto, 5 de 5, y para tener alternativas hace falta que las muestras difieran, sin llegar a un extremo que rompa el texto (a temperatura 1.5 con top-p 1.0 ya apareció una variación de formato). Con 5 ejecuciones es evidencia, no prueba.

9. Parte 3 — prompting estructurado

Se compararon cuatro variantes sobre `propietario_economico` (el mismo modelo propietario de la Parte 1 con salida estructurada de esquema estricto), los mismos 10 casos y 5 ejecuciones (200 llamadas): zero-shot (solo la instrucción); few-shot con tres ejemplos propios; chain-of-thought (se pide razonar paso a paso y dar el JSON en la última línea; se verifica solo la respuesta final, pero la salida completa se conserva); y salida estructurada con un esquema JSON estricto que impone la API. La respuesta completa de cada variante para un caso está en el Anexo C. Los costos de entrada y de salida se reportan por separado porque no cuestan lo mismo.

Variant e	Exactitud	Formato válido	Parse rate	Tokens entrada	Tokens salida	Costo entrada	Costo salida	Costo total	Frente a zero-shot
zero_shot	1.00	1.00	1.00	57.1	6.0	\$0.00043	\$0.00018	\$0.00061	1.0×
few_shot	1.00	1.00	1.00	133.1	6.0	\$0.00100	\$0.00018	\$0.00118	1.9×
cot	1.00	1.00	1.00	82.1	133.6	\$0.00062	\$0.00401	\$0.00462	7.6×
estructur ed	1.00	1.00	1.00	80.1	5.0	\$0.00060	\$0.00015	\$0.00075	1.2×

Las cuatro variantes tienen exactitud 1.00 y formato válido 1.00, así que todo el contraste es de costo. Few-shot cuesta **1.9×** el costo de zero-shot solo por los tokens de entrada de los ejemplos (133.1 frente a 57.1). Chain-of-thought cuesta **7.6×** porque produce 133.6 tokens de salida frente a 6.0 (unas 22 veces más) sin mejorar la exactitud, y los tokens de salida se cobran más caros que los de entrada (USD 0.6 frente a USD 0.15 por millón). La salida estructurada cuesta 1.2× y es la más corta en salida (5.0 tokens), pero gasta más entrada que zero-shot (80.1 frente a 57.1) pese a tener un prompt más corto; lo más probable es que el esquema cuente como entrada, aunque eso se infiere de las cifras y no está en la documentación consultada.

Qué garantiza el proveedor y qué se verificó (variante 4). Se usó salida estructurada con esquema estricto (`response_format` de tipo `json_schema` con `strict`). Pedir JSON en el prompt no garantiza nada (zero-shot cumplió el formato en 1.00 de las llamadas, pero por suerte y no por garantía); el modo JSON garantiza que parsea; el esquema estricto garantiza además las claves, los tipos y un valor del enum. Ninguno garantiza que el valor sea el correcto, y esa comprobación es propia: sobre 50 llamadas se verificó `parse_rate` 1.00, formato válido 1.00 y exactitud 1.00. Hay además una prueba automática con un JSON válido y una categoría equivocada, que el esquema deja pasar.

Conclusión: cuándo usaría cada técnica. Zero-shot es el punto de partida y aquí bastó: exactitud 1.00 con el menor costo, así que lo usaría mientras funcione. Few-shot lo usaría cuando el formato o el criterio de las etiquetas sea difícil de describir con palabras y el modelo lo confunda: cuesta 1.9× solo por los ejemplos (133.1 tokens de entrada frente a 57.1) y ese costo se repite en cada llamada, de modo que conviene con prompts cortos, con volumen bajo o con caché del prefijo. Chain-of-thought lo reservaría para tareas de varios pasos, como aritmética, lógica o extracción con reglas encadenadas, donde zero-shot falla;

aquí costó $7.6\times$ sin ganancia, porque produjo 133.6 tokens de salida en lugar de 6.0. La salida estructurada la usaría siempre que otro programa consuma la respuesta: garantiza claves, tipos y valores del enum, y costó $1.2\times$ lo de zero-shot; su límite es que no garantiza el valor correcto, que se verificó aparte (1.00 de exactitud). En producción combinaría zero-shot o few-shot con esquema estricto y mediría la exactitud sobre un conjunto etiquetado antes de añadir razonamiento, porque cada técnica cambia el costo por llamada de forma distinta: los ejemplos añaden entrada, el razonamiento añade salida y el esquema añade un poco de entrada. Todo esto descansa en 10 casos fáciles y un solo modelo; no permite afirmar que las diferencias sean nulas en tareas difíciles.

10. Parte 4.a — esfuerzo de razonamiento

Modelo `openai_razonamiento` (gpt-5.6-luna, USD 0.2 / 1.2 por millón, precio verificado el 2026-09-18). Se averiguó qué niveles acepta, como en la Parte 2.a, y se corrieron los 10 casos en los 5 niveles aceptados (3 ejecuciones por nivel, 150 llamadas correctas). El costo es el de los tokens de salida totales, con el razonamiento incluido, al precio de la fila; los tokens de razonamiento salen de `usage.completion_tokens_details.reasoning_tokens` y los visibles son el total menos esos.

Esfuerzo	Llamadas	Exactitud	Tokens de razonamiento	Salida visible	Salida total	Latencia media (s)	Costo por llamada	Costo del nivel
none	30	1.00	0.00	8.0	8.0	0.85	\$0.0000208	\$0.000625
low	30	1.00	0.00	8.0	8.0	0.78	\$0.0000208	\$0.000625
medium	30	1.00	0.00	8.0	8.0	0.76	\$0.0000208	\$0.000625
high	30	1.00	0.37	8.2	8.6	0.83	\$0.0000215	\$0.000645
xhigh	30	1.00	1.57	8.8	10.4	0.80	\$0.0000237	\$0.000710
max	1	—	—	—	—	—	—	—

Hay tres hallazgos. Primero, **max no existe para este modelo**: la tabla del curso lo lista, pero la API responde «Unsupported value: 'reasoning_effort' does not support 'max' with this model. Supported values are: 'none', 'low', 'medium', 'high', and 'xhigh'». Segundo, **la exactitud fue 1.00 en todos los niveles**, así que el aumento de exactitud entre niveles es cero y el costo por punto de exactitud no está definido. Tercero, el modelo casi no razonó con estos tickets: solo **5 de 150 llamadas** usaron tokens de razonamiento, todas en `high` o `xhigh` y solo en los casos `case_04` y `case_10`.

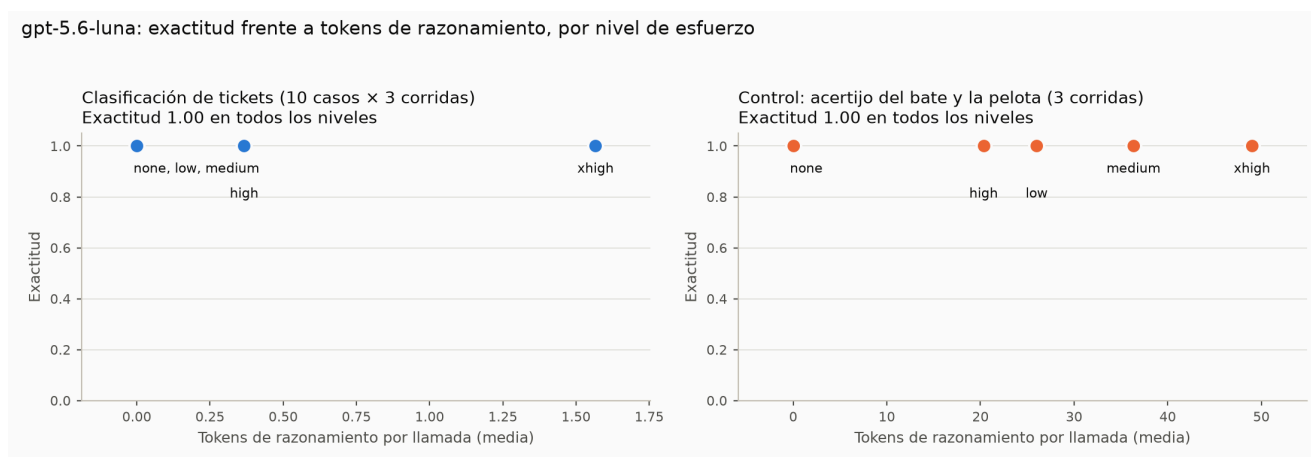


Figura 5. Exactitud frente a tokens de razonamiento medios por nivel de esfuerzo: tickets a la izquierda y acertijo de control a la derecha. Los niveles con las mismas coordenadas comparten una sola marca.

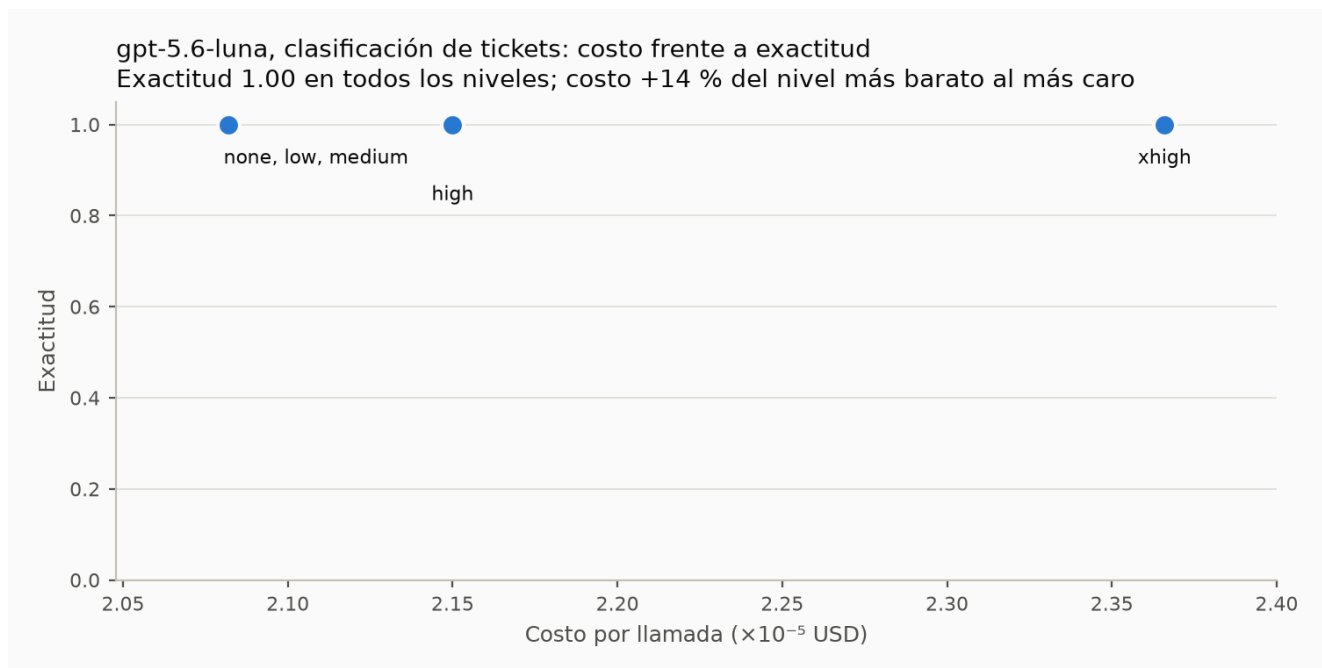


Figura 6. Costo por llamada frente a exactitud en la clasificación de tickets. El eje vertical va de 0 a 1 para no exagerar diferencias.

Presupuesto: estimado frente a medido. El enunciado pide estimar antes de lanzar, con el precio de la fila y su fecha, y esa estimación va en el informe. Con su supuesto (1000 tokens de entrada, 600 de salida visible y una salida total de 1500 / 3000 / 7000 tokens en los niveles bajo / medio / alto) y el precio de USD 0.2 / 1.2 por millón:

Nivel	Razonamiento supuesto	Salida total supuesta	Costo por llamada estimado	Razonamiento medido	Costo por llamada medido	Medido / estimado
low	900	1500	\$0.00200	0.00	\$0.0000208	0.010×
medium	2400	3000	\$0.00380	0.00	\$0.0000208	0.005×
high	6400	7000	\$0.00860	0.37	\$0.0000215	0.002×

Para el diseño del enunciado (3 niveles $\times$ 10 casos más 6 llamadas contaminadas) la estimación es de **\$0.176**; el gasto medido de toda la Parte 4 (4.a y 4.b, con más niveles y el control) fue de **\$0.0080**, unas 22 veces menos. Hay que ser honesto con el orden: esta estimación se calculó al redactar el informe y no antes del barrido; antes de él solo se hizo un piloto de 1 caso en los 6 niveles, que ya mostró 0 tokens de razonamiento, y un `--dry-run` que no puede estimar los tokens de razonamiento. No se corrió un nivel completo antes de los tres, como pide la regla 2, porque el piloto ya mostraba que el contador estaba en cero. La medición se separa del supuesto justo en el término que el enunciado marca como el menos predecible: en el nivel alto se suponían 6400 tokens de razonamiento y se midieron 0.37. Es un hallazgo de esta parte.

Con un piloto de un caso los cinco niveles dieron cero tokens de razonamiento, y eso podía significar que el parámetro no hacía nada (el caso «aceptado y no actúa»). Se diagnosticó mirando el contador y no la prosa de la respuesta, con un acertijo de control (el del bate y la pelota, 3 ejecuciones por nivel):

Esfuerzo	Ejecuciones	Razonamiento (media)	Salida total (media)	Respuesta correcta
none	3	0.0	4.0	1.00
low	3	26.0	36.0	1.00
medium	3	36.3	46.3	1.00
high	3	20.3	30.3	1.00
xhigh	3	49.0	59.0	1.00

El dial sí actúa: de 0 tokens de razonamiento en `none` a 49 en `xhigh`. No es monótono entre niveles intermedios (26 / 36 / 20 / 49 en `low`, `medium`, `high`, `xhigh`), pero con tres ejecuciones por nivel solo se sostiene la tendencia general. El acertijo se resolvió bien incluso sin razonamiento.

Conclusión. Mi punto de rendimientos decrecientes está en el primer escalón: con exactitud 1.00 desde `none`, ningún nivel compró exactitud, así que no puedo decir «a partir de tal nivel, cada punto adicional me costó X USD»; el costo por punto de exactitud no está definido porque el aumento de exactitud es cero. Lo que sí se puede decir con cifras es lo que costó no ganar nada: de `none` a `xhigh` el costo por llamada subió de \$0.0000208 a \$0.0000237 (14 %), con saltos de \$0.0000007 de `medium` a `high` y de \$0.0000022 de `high` a `xhigh`. La medición también se separó del supuesto del enunciado: en el nivel alto se suponían 6400 tokens de razonamiento y se midieron 0.37. La curva es plana, y eso es un resultado: para esta tarea el nivel se elige por medición y no se sube por defecto, así que usaría `none`. El control con el acertijo muestra que el dial funciona, de modo que la curva plana describe a la tarea y no al parámetro; no se puede extender a tareas que sí exijan razonar.

11. Parte 4.b — el caso donde pensar más podría hacer daño

Se escribieron tres casos fáciles con una trampa cada uno: una **distracción** (una anécdota larga con palabras de pagos y facturas, pero lo que se pide es cambiar el correo del perfil), un **marco engañoso** (el usuario cree que es un fallo técnico, pero pide el reembolso de un cobro duplicado) y una **correlación espuria** (menciona «premium», «pago» y «factura», pero el problema es que la aplicación se cierra; sigue el ejemplo del plan). La categoría correcta es inequívoca en los tres, y una prueba automática comprueba que cada ticket trae señales de una categoría equivocada. Se corrieron en el nivel más bajo (`none`) y en el más alto (`xhigh`) de esfuerzo, con 10 ejecuciones por celda; `low` y `high` se corrieron antes, por el plan inicial, y se dejan como niveles intermedios.

Caso	Trampa	Esfuerzo	Ejecuciones	Aciertos	Razonamiento (media)	Razonamiento (máx.)
contaminated_01	distracción	none	10	1.00	0.0	0
contaminated_01	distracción	low	10	1.00	0.0	0
contaminated_01	distracción	high	10	1.00	1.6	16
contaminated_01	distracción	xhigh	10	1.00	4.3	18
contaminated_02	marco engañoso	none	10	1.00	0.0	0
contaminated_02	marco engañoso	low	10	1.00	0.0	0
contaminated_02	marco engañoso	high	10	1.00	4.5	9
contaminated_02	marco engañoso	xhigh	10	1.00	7.8	19
contaminated_03	correlación espuria	none	10	1.00	0.0	0
contaminated_03	correlación espuria	low	10	1.00	0.0	0
contaminated_03	correlación espuria	high	10	1.00	9.2	19
contaminated_03	correlación espuria	xhigh	10	1.00	22.2	28

En los dos niveles extremos, 60 de 60 llamadas fueron correctas, y en total 120 de 120: pensar más no empeoró nada. Las predicciones fueron idénticas entre ejecuciones. El razonamiento sí crece con el esfuerzo en los tres casos, pero es minúsculo: el máximo fue de 28 tokens en una llamada. En cada nivel con razonamiento el caso con más tokens fue el de la correlación espuria y el de menos el de la distracción; con 10 ejecuciones por celda es un indicio, no una conclusión.

Contraste con Gema et al. (2025). El artículo *Inverse Scaling in Test-Time Compute* (TMLR, 12/2025) reporta que más razonamiento puede empeorar la exactitud: los modelos Claude se distraen cada vez más con información irrelevante, los modelos de la serie o de OpenAI resisten los distractores pero se sobreajustan al marco del problema, y los modelos pasan de priors razonables a correlaciones es-

purias. Este experimento no lo reproduce, pero tampoco lo contradice. Las tareas del artículo son otras (conteo con distractores, regresión con rasgos espurios, deducción, riesgos de IA), más difíciles, con presupuestos de razonamiento mucho más largos y otros modelos; aquí hay tres casos fáciles con exactitud del 100 % y decenas de tokens de razonamiento, así que ni siquiera se llega al régimen que estudian. Para detectar un efecto inverso harían falta casos que el modelo pueda fallar. **Tres casos no demuestran una ley**, ni a favor ni en contra; lo que sí deja el ejercicio es el hábito: el nivel de esfuerzo se elige por tarea y se mide, no se sube por defecto.

12. Parte 5 — reflexión teórica

1. Atención en transformers (query, key, value).

En cada capa, cada token se convierte en tres vectores aprendidos: una consulta (query), una clave (key) y un valor (value). La consulta de un token expresa qué información busca; la clave de otro, qué información ofrece. El producto punto entre consulta y clave, escalado y pasado por una softmax, da pesos de atención: cuánto debe mirar el token actual a cada uno de los demás. Con esos pesos se promedian los valores, que son el contenido que de verdad se transmite, y el resultado pasa a ser la nueva representación del token. Así cada posición reúne información del contexto sin importar la distancia. Todo esto ocurre en varias cabezas en paralelo, cada una libre de fijarse en relaciones distintas, y en un modelo generativo una máscara causal impide mirar tokens futuros.

2. Modelo base frente a modelo alineado. Salida de GPT-2 base (Parte 0.c, caso 1, sin editar):

```
I was charged twice for the same subscription.  
I was charged twice for the same subscription.  
I was charged twice for the same subscription.  
I was charged twice for the same subscription.  
I was charged twice for the same subscription.  
I was charged
```

Un modelo base solo aprendió a continuar texto: dado un prefijo, produce lo que en su corpus solía venir después. Por eso, ante el prompt de clasificación, repitió el ticket y no clasificó nada; nada en su entrenamiento le dice que aquello era una orden. Un modelo alineado pasó además por ajuste con instrucciones y con preferencias humanas, y eso convierte esa capacidad en «hacer lo que se pide, en el formato pedido». Para las aplicaciones de este curso el base no sirve directamente porque necesitan respuestas obedientes y parseables: GPT-2 dio 0 de 10 respuestas parseables y los tres modelos alineados de la Parte 1 acertaron al menos 10 de 10. Hay un límite: GPT-2 también es mucho más pequeño y antiguo, así que la comparación no separa alineamiento de tamaño.

3. Chain-of-thought, tokens, exactitud y costo.

En la Parte 3 la exactitud fue 1.00 en las cuatro variantes, pero chain-of-thought produjo 133.6 tokens de salida frente a 6.0 y costó 7.6× lo de zero-shot. Los tokens de salida se cobran más caros que los de entrada (USD 0.6 frente a 0.15 por millón), así que el gasto crece con la longitud del razonamiento y no con la dificultad de la pregunta. Vale la pena cuando la tarea exige pasos intermedios y el modelo sin razonar falla;

aquí no fallaba, y todo el gasto extra fue sobrecarga. Tampoco conviene si la latencia importa, porque cada token visible se genera en secuencia. Los modelos de razonamiento hacen lo mismo por dentro y también lo facturan como salida. La regla que queda es medir: pagar por razonar solo donde la exactitud sube.

4. Un parámetro rechazado: qué gana el proveedor y qué pierde quien lo usaba.

En la matriz, `openai_razonamiento` rechaza `temperature` y `top_p` salvo su valor por defecto, y `top_k` en ambos modelos de OpenAI. Esos parámetros reescalan o recortan la distribución antes de muestrear; quien los usaba perdía el control directo de la variación y la vía para pedir una salida casi determinista. Lo que gana el proveedor no está documentado, y lo que sigue es una hipótesis: un modelo de razonamiento se entrena y se sirve con un muestreo fijo, y dejar libres los diales multiplica las configuraciones por soportar y permite combinaciones que degradan el razonamiento. Lo que sustituye a la palanca es otra: el esfuerzo controla cuánto se computa, no cuánta aleatoriedad hay; para variar salidas se repite y se vota, y para estabilizarlas se restringe la forma con un esquema. En clasificación se pierde poco: exactitud 1.00 en las 15 celdas de la 2.b.

5. Fidelidad de una traza de razonamiento completa (Lanham et al., 2023).

No necesariamente. Lanham et al. (2023) intervienen sobre el razonamiento escrito, agregándole errores o parafraseándolo, y miden si la respuesta cambia: si no cambia, el texto no era la causa. Encuentran que la dependencia del razonamiento varía mucho según la tarea y que los modelos más grandes y capaces producen razonamiento menos fiel en la mayoría de ellas. Ver la traza completa daría entonces una narración plausible, no una explicación verificada. Para auditar un sistema en producción, leer la traza no basta: hay que ponerla a prueba con intervenciones y registrar cuánto se razona, que es lo que sí se mide (Parte 4). Aquí ni siquiera llegó el texto, solo el contador, y en la Parte 3 la variante sin razonamiento acertó igual, lo que sugiere que en esa tarea no era necesario.

13. Conclusiones

Cada parte tiene su propia conclusión arriba. En conjunto:

1. **La exactitud no discriminó.** Tres modelos, cuatro variantes de prompt, 15 combinaciones de muestreo, cinco niveles de esfuerzo y tres casos contaminados dieron exactitud igual o casi igual al 100 %. Lo que cambió entre condiciones fue el costo (`gpt-5.6-luna` 1.7 veces el de `gpt-4o-mini` en la Parte 1; `chain-of-thought` 7.6× el de `zero-shot`; `xhigh` 14 % más que `none`). El diseño tiene un techo: con tickets fáciles no hay margen para ver mejoras ni empeoramientos.
2. **Lo declarado no es lo observado.** Un prefijo «de alta confianza» que no lo era, un nivel `max` inexistente, y un modelo cuyos parámetros de muestreo la tabla declara como no presentes y que aceptan solo el valor por defecto. Medir cada supuesto antes de barrer evitó gastar en experimentos que no habrían probado nada.
3. **Un parámetro puede actuar y no importar.** En texto libre `temperature` y `top-p` sí cambian la salida; en clasificación no cambian la respuesta, porque la decisión es un token dominante.
4. **El razonamiento existe pero casi no se usa en tareas fáciles.** El dial de esfuerzo funciona (se comprobó con un acertijo) y su costo es pequeño aquí, pero no compró exactitud, ni siquiera frente a las trampas de la 4.b.

Lo que no se puede concluir. Que los modelos «sean equivalentes» o que el muestreo, el razonamiento o `chain-of-thought` «no sirvan»: solo que no aportan en este problema, con estos modelos y a esta fecha.

Los tickets son sintéticos y fáciles. Las cinco ejecuciones por ajuste y las diez por celda dan evidencia pero no pruebas; las desviaciones observadas en el barrido son eventos únicos. La estimación de presupuesto de la Parte 4 se hizo después del barrido. Los precios de los modelos de la tabla se leyeron de ella, pero los resultados de APIs pueden cambiar con el tiempo. Para ver diferencias de verdad harían falta casos que los modelos puedan fallar.

14. Reproducibilidad

Se necesita Python 3.11, [uv](#), claves en `.env` (`OPENAI_API_KEY`; el archivo está excluido por `.gitignore`) y, para el modelo local, Ollama con `qwen3:1.7b`. La Parte 0 descarga GPT-2 (unos 550 MB) y corre en CPU sin API.

```
uv venv --python 3.11
uv pip install -r requirements.txt
cp .env.example .env          # completar OPENAI_API_KEY
uv run pytest                  # pruebas
uv run scripts/run_all.py --all --dry-run  # cuenta y estima sin llamar a APIs
uv run scripts/run_all.py --all          # o una parte: --part 0, 1, 2a, 2b, 3, 4a, 4b
uv run scripts/aggregate_results.py
uv run scripts/generate_plots.py
uv run scripts/build_report.py && uv run scripts/build_pdf.py && uv run
scripts/build_docx.py
```

Costo de las llamadas a APIs (todo el proyecto):

Parte	Llamadas	Errores (rechazos esperados)	Costo (USD)
Parte 0 (GPT-2 local)	25	0	\$0.0000
Parte 1	43	0	\$0.0110
Parte 2.a	90	19	\$0.0177
Parte 2.b	950	0	\$0.0091
Parte 3	200	0	\$0.0072
Parte 4.a	166	1	\$0.0040
Parte 4.b	120	0	\$0.0040

Todo está versionado. `outputs/raw/results.jsonl` (una fila por llamada, con las 25 generaciones locales de la Parte 0 incluidas) es la fuente de la que salen las tablas, las gráficas de la Parte 4 y este texto, así que se pueden regenerar sin llamar a ninguna API (`aggregate_results.py`, `generate_plots.py`, `build_report.py`). Volver a producir el archivo desde cero exige volver a correr los experimentos; los que llaman a APIs son reanudables y el costo total fue \$0.053. Las salidas de la Parte 0 (`part0*.json`) se reconstruyen a partir de esas filas y, al repetir la ejecución, resultaron idénticas. No hay claves en el repositorio; se revisó el historial de git y los archivos con una búsqueda de cadenas con formato de clave y no apareció ninguna.

Anexo A — Salidas crudas de la Parte 0.b

Las salidas de los anexos se pegan tal cual salieron del modelo; lo único que se recorta son los saltos de línea del principio y del final de cada bloque.

A.1 — `do_sample=False`, **temperature 0.2** (40 tokens nuevos):

```
problem.
```

The customer support ticket about an unexpected problem.
The customer support ticket about an unexpected problem.
The customer support ticket about an unexpected problem.
The customer support

A.2 — `do_sample=False`, `temperature 1.5`:

problem.
The customer support ticket about an unexpected problem.
The customer support ticket about an unexpected problem.
The customer support ticket about an unexpected problem.
The customer support

A.3 — `do_sample=True`, `top_k=1`, **semilla 42** (las otras cuatro ejecuciones, con semillas 43–46, dieron exactamente la misma salida; los ids generados coinciden con los de greedy en 5 de 5 ejecuciones):

problem.
The customer support ticket about an unexpected problem.
The customer support ticket about an unexpected problem.
The customer support ticket about an unexpected problem.
The customer support

A.4 — Degeneración: `do_sample=False`, `100 tokens`, `sin editar`:

problem.
The customer support ticket about an unexpected problem.
The customer support ticket about an unexpected problem.
The customer support ticket about an unexpected problem.
The customer support ticket about an unexpected problem.
The customer support ticket about an unexpected problem.
The customer support ticket about an unexpected problem.
The customer support ticket about an unexpected problem.
The customer support ticket about an unexpected problem.
The customer support ticket about an unexpected problem.
The customer support ticket about an unexpected problem

Anexo B — Salidas crudas de la Parte o.c (GPT-2 base con el prompt de clasificación)

El prompt es el mismo de las Partes 1 a 4; el ticket va al final. Ejemplo, caso 1:

```
Classify the support ticket into exactly one category:

- billing
- technical
- account

Return only a JSON object of the form {"category": "<category>"}.

Ticket:
I was charged twice for the same subscription.
```

case_01 (categoría esperada: `billing`; parseable: no):

```
I was charged twice for the same subscription.

I was charged twice for the same subscription.

I was charged twice for the same subscription.

I was charged twice for the same subscription.

I was charged twice for the same subscription.

I was charged
```

case_02 (categoría esperada: `technical`; parseable: no):

```
The application crashes every time I try to upload a file.

The application crashes every time I try to upload a file.

The application crashes every time I try to upload a file.

The application crashes every time I try to upload a file.

The application
```

case_03 (categoría esperada: `account`; parseable: no):

```
I can't access my account.

I can't access my account.

I can't access my account.

I can't access my account.

I can't access my account.

I can't access my account.
```

I can't access

case_04 (categoría esperada: `billing`; parseable: no):

Example:

\$1 = \$1.00

\$2 = \$2.00

\$3 = \$3.00

\$4 = \$4.00

\$5 = \$5.00

\$6 = \$6.00

case_05 (categoría esperada: `technical`; parseable: no):

The following example shows how to create a ticket for a specific category.

```
package main import ( "fmt" "log" "net/http" "log/http" "net/http/http.core"
"net/http/http.core.core.core
```

case_06 (categoría esperada: `account`; parseable: no):

I want to change the email address associated with my profile.

I want to change the email address associated with my profile.

I want to change the email address associated with my profile.

I want to change the email address associated with my profile.

I want

case_07 (categoría esperada: `billing`; parseable: no):

I would like to know if the refund is valid.

I would like to know if the refund is valid.

I would like to know if the refund is valid.

I would like to know if the refund is valid.

I would like to know if

case_08 (categoría esperada: `technical`; parseable: no):

The export to JSON button does nothing when I click it.

The export to CSV button does nothing when I click it.

The export to JSON button does nothing when I click it.

The export to JSON button does nothing when I click it.

The export

case_09 (categoría esperada: `account`; parseable: no):

If you want to delete the data, you can use the following command:

```
$ curl -X POST -d '{"category": "my-category"}' -H 'Content-Type: application/json' -d '{"category": "my-category"}'
```

case_10 (categoría esperada: `billing`; parseable: no):

Example:

\$1 = \$1.99

\$2 = \$2.99

\$3 = \$3.99

\$4 = \$4.99

\$5 = \$5.99

\$6 = \$6.99

Anexo C — Respuestas completas de la Parte 3 (un caso por variante)

C.1 — `zero_shot` (caso `case_01`, ejecución 1; 53 tokens de entrada, 6 de salida):

```
{"category": "billing"}
```

C.2 — `few_shot` (caso `case_01`, ejecución 1; 129 tokens de entrada, 6 de salida):

```
{"category": "billing"}
```

C.3 — `cot` (caso `case_01`, ejecución 1; 78 tokens de entrada, 118 de salida):

Let's analyze the customer's issue step by step:

1. The customer mentions being charged twice, which indicates a problem related to charges or payments they have made.
2. This issue does not pertain to accessing or using a service (which would indicate a technical problem) and does not involve the management or setup of their user profile or account information.
3. The nature of the issue revolves around billing because it directly relates to a misunderstanding or error in charges.

Given these points, the correct category for this support ticket is billing.

Final answer:

```
'''json
```

```
{"category": "billing"}  
'''
```

C.4 — structured (caso case_01, ejecución 1; 76 tokens de entrada, 5 de salida):

```
{"category":"billing"}
```

Referencias

- Holtzman, A. et al. (2020). *The Curious Case of Neural Text Degeneration*.
<https://arxiv.org/abs/1904.09751>
- Gema, A. P. et al. (2025). *Inverse Scaling in Test-Time Compute*. TMLR, 12/2025.
<https://arxiv.org/abs/2507.14417>
- Lanham, T. et al. (2023). *Measuring Faithfulness in Chain-of-Thought Reasoning*.
<https://arxiv.org/abs/2307.13702>